Terraform Fundamentals

Instructor: Dyego Souza
Findmore Academy
https://www.linkedin.com/in/dyegodesouza/
ramehx@yahoo.com.br

SESSION 2: WORKING WITH VARIABLES AND RESOURCES

Structure of .tf Files

Terraform blocks

Terraform uses several types of blocks to describe infrastructure as code. Each block has a specific purpose and plays a fundamental role in configuring the cloud environment.

Example of blocks: Provider, Resource, variables, locals, output, module, etc.

Provider Definitions

Providers specify the cloud or on-premise platforms managing the infrastructure resources

Infrastructure Resources

Resources in .tf files define the infrastructure components to be created and managed.

Variables and Outputs

Variables allow customization, while outputs export useful information from the configuration.

mycode.tf

C: > Users > rameh > Downloads > mycode.tf > ...

```
# Provider declaration
provider "azurerm" {
  features {}
}

# Input variables
variable "location" {
  description = "Azure region to deploy resources"
  type        = string
  default     = "East US"
}

locals {
  rg_name = "rg-terraform-demo"
}

variable "storage_accounts" {
  description = "List of Storage Account names"
  type        = list(string)
  default     = ["stterraform01", "stterraform02"]
}

# Resource Group
resource "azurerm_resource_group" "rg" {
  name     = local.rg_name
  location = var.location
}

# Storage Accounts using loop (for_each)
resource "azurerm_storage_account" "storage" {
  for_each = toset(var.storage_accounts)

  name                     = each.value
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"

  tags = {
    environment = "demo"
  }
}
```

FINDMORE ACADEMY

Resource block

- The resource block is used to define the resources that will be provisioned in the cloud provider.
- It includes resource-specific attributes, such as name, location, and detailed settings.
- Resource blocks are declared in the following format:

```
Resource "type" "name"{
  parameter_a = "abcdefgh"
  parameter_b = 123456
}
```

```
resource "azurerm_virtual_network" "example" {
  name                = "example-network"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name
}
```

- **Resource**: keyword
- **Type**: type of resource. E.g.: azurerm_resource_group
- **Name:** arbitrary and unique name for internal reference of terraform
- **Parameters:** attributes that might vary from resource to resource. Some are optional and some are mandatory

FINDMORE ACADEMY

Terraform Settings Block

terraform { }

- Defines **global settings** for a Terraform configuration.
- Declares **required providers** and their versions.
- Configures the **backend** for state storage and locking.
- Sets Terraform CLI constraints (minimum required version).
- **Common elements:**
- required_version
- required_providers
- backend

"Which provider and version do I need?

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }

    azurecaf = {
      source  = "aztfmod/azurecaf"
      version = "~> 1.2"
    }
  }
  required_version = ">= 1.3.0"

  backend "local" {
    path = "relative/path/to/terraform.tfstate"
  }
}
```

FINDMORE ACADEMY

Interno

27

Provider Block

provider { }

- Configures **how Terraform connects** to a specific platform or API.
- Defines **authentication, subscription/account, and regional settings**.
- Uses provider plugins declared in required_providers.
- Can be defined multiple times using **aliases**.

Typical provider configuration includes:

- Credentials and authentication method
- Subscription / account context
- Region or location
- Provider-specific features

Example of providers:

Azure, AWS, GCP, etc.

"How do I connect and use that provider?"

```
# Provider declaration
provider "azurerm" {
  features {}
  subscription_id = "subscription_id"
  tenant_id       = "tenant_id"
  client_id       = "principal_id"
  alias           = "azurerm-1"


}
```

FINDMORE ACADEMY

Interno

28

Terraform Version Constraints

- Specify the minimum and/or maximum Terraform version your configuration supports.
- Ensures **compatibility** with the configuration and provider features.
- Declared inside the terraform {} block
- Helps prevent breaking changes in team environments and CI/CD pipelines.
- Supports operators:

  = : exact version

  >= : minimum version

  <= : maximum version

  ~> : compatible version, e.g.,

  ~> 1.5 → >= 1.5.0 and < 2.0.0

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }

    azurecaf = {
      source  = "aztfmod/azurecaf"
      version = "~> 1.2"
    }
  }
  required_version = ">= 1.3.0"
```

> Terraform version

```
PS C:\Users\rameh\kind> terraform version
Terraform v1.9.8
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.14.3. You can update by downloading from https://www.terraform.io/downloads.html
PS C:\Users\rameh\kind>
```

FINDMORE ACADEMY

Interno

29

Variable blocks: Input

Variables {}

- Serve as parameter for configuration of resources on your Terraform code
- Can have the following elements:
  - **Default values**: Will be used if user does not set any value for the variable
  - **Descriptions**: Brief explanation of the purpose of the variable (best practice)
  - **Type constraints**: specify which type is accepted for the variable

Primitive Types:

- string
- Number
- bool

```
# Input variable types
variable "location" {
  description = "Azure region for resource deployment"
  type        = string
  default     = "East US"
}


variable "instance_number" {
  description = "Number of container instances"
  type        = number
  default     = 3
}


variable "boolean_example" {
  description = "Example of boolean var"
  type        = bool
  default     = false
}
```

https://spacelift.io/blog/terraform-map-variable

FINDMORE ACADEMY

Interno

30

Variable blocks: Input

Collection Types

Accept any type of elements, as long as every elements is the same type

- **List**: ordered collection
- **Map**: key-value mapping. Support:
  - Map(string)
  - Map(number)
  - Map(list)
  - Map(object({...}))
  - Etc.

Complex Types

Each element can have its own type

- **Tuple:** a sequence of elements with fixed length.
- **Object:** a collection of structured, mixed type elements

```
# Example of list
variable "environment" {
  description = "Environment name"
  type        = list(string)
  default     = ["dev", "prod", "qa"]
}

variable "tuple_example" {
  description = "Sequence of elements with different types"
  type        = tuple([string, number, bool])
  default = ["a", 15, true]
}

# object example
variable "server_config" {
  description = "Environment-specific configuration"
  type = object({
    name          = string
    instance_type = string
    max_size      = number
  })
}

# Example of map of objects
variable "env_config" {
  description = "Environment-specific configuration"
  type = map(object({
    instance_type = string
    min_size      = number
    max_size      = number
  }))

  default = {
    dev = {
      instance_type = "t3.small"
      min_size      = 1
      max_size      = 2
    }
    prod = {
      instance_type = "t3.large"
      min_size      = 3
      max_size      = 6
    }
  }
}
```

Interno

Variable blocks: Locals

Locals {}

- Used to define internal computed values within a module.
- Help reduce code duplication and simplify references.
- Can be called on the code by **local.variable_name**

```
# Local variables
locals {
  resource_group_name = "rg-${var.environment}-rg"

  region = "northeurope"

  tags = {
    owner = "Infra team"
    contact = "infra@corp.com"
    environment = var.environment
  }
}
```

FINDMORE ACADEMY

Interno

Variable Interpolation

- Interpolation in Terraform allows you to dynamically insert values from variables, data sources, or function calls into your configuration strings.
- Interpolation is done using the ${...} syntax.
- Ex: ${var.region} inserts the value of the variable named **region**

```
variable "environment" {
  default = "prod"
}

resource "azurerm_resource_group" "rg_prod" {
  name     = "rg-${var.environment}"
  location = "westeurope"
}
```

FINDMORE ACADEMY

Interno

.tfvars files

- **.tfvars** files are used to assign values to input variables.
- Separate configuration data from Terraform code.
- Commonly used to manage environment-specific values (dev, test, prod).
- Terraform automatically loads when named:
  - terraform.tfvars
  - *.auto.tfvars
- Terraform follows this precedence order for variables:

1. CLI -var
2. .tfvars / *.auto.tfvars
3. Default values (variables.tf)

Variables can be set using command line:

> terraform apply –var="instance_number=5"

Or with configuration files ***.tfvars** or ***.auto.tfvars**

> terraform apply -var-file="prod.tfvars"

terraform.tfvars

C: > terraform > terraform.tfvars > ...

```
location        = "East US"
environment     = "prod"
instance_number = 5
```

FINDMORE ACADEMY

Interno

Output Block

Output { }

- output blocks expose information from Terraform configurations or modules.
- Useful for sharing values between modules or displaying important info after apply.

Can define:

- **value** → the actual output
- **description** → optional description
- **sensitive** = true → hides secrets

```
output "db_instance_name" {
  value = aws_db_instance.main.name
  description = "DB Instance Name"
}

output "database_password" {
  value     = aws_db_instance.main.password
  sensitive = true
}
```

```
~/Devops/terraform test$ terraform apply tfplan
azurerm_resource_group.rg: Creating...
azurerm_resource_group.rg: Still creating... [00m10s elapsed]
azurerm_resource_group.rg: Creation complete after 14s [id=/subscriptions/b0c2c839-bd2d-4f80-bfe9-52b2828de1e4/resourceGrou
azurerm_virtual_network.vnet: Creating...
azurerm_virtual_network.vnet: Creation complete after 6s [id=/subscriptions/b0c2c839-bd2d-4f80-bfe9-52b2828de1e4/resourceGr
rs/Microsoft.Network/virtualNetworks/vnet-demo]
azurerm_subnet.subnet: Creating...
azurerm_subnet.subnet: Creation complete after 9s [id=/subscriptions/b0c2c839-bd2d-4f80-bfe9-52b2828de1e4/resourceGroups/rg
osoft.Network/virtualNetworks/vnet-demo/subnets/subnet-demo]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

resource_group_location = "westeurope"
resource_group_name = "rg-demo"
```

Interno

Module block

- Modules are **reusable, composable Terraform configurations**.
- Allow encapsulation of resources for better maintainability.
- Can be sourced from:
  - Local paths (./modules/...)
  - Git repositories
  - Terraform Registry

```
module "storage_account" {
  source              = "./modules/storage"
  location            = var.location
  resource_group_name = local.rg_name
}
```

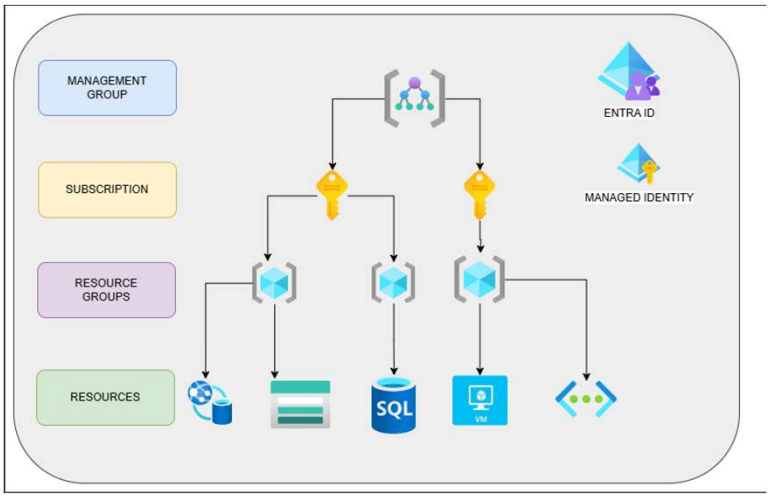
```
Dev
  ├── jenkins-using-packer.tfvars
  ├── main.tf
  └── variable.tf
Module
  ├── Ec2andLoadBalancer
  │   ├── main.tf
  │   └── variable.tf
  ├── ECS
  │   ├── main.tf
  │   ├── output.tf
  │   └── variable.tf
  ├── Networking
  │   ├── main.tf
  │   ├── output.tf
  │   └── variable.tf
  └── SecurityGroup
      ├── main.tf
      ├── output.tf
      └── variable.tf
```

Fonte: https://medium.com/xebia-engineering/best-practices-to-create-organize-terraform-code-for-aws-2f4162525a1a

Interno

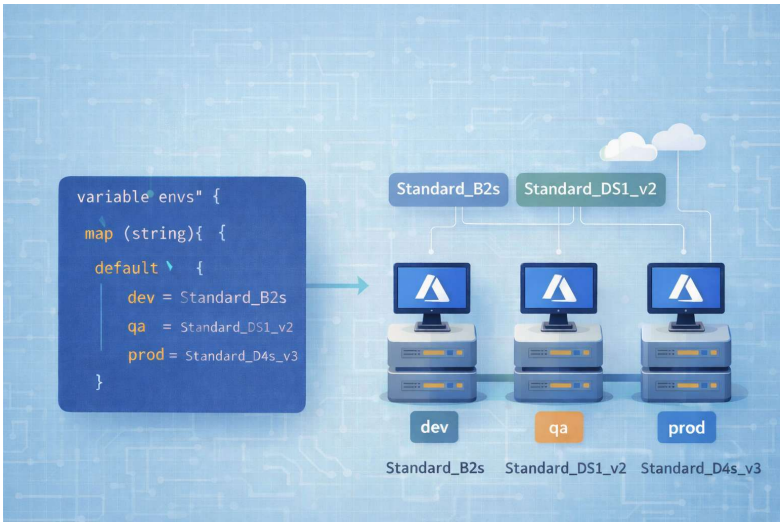
Azure Overview

- Management Groups
- Subscriptions
- EntraID
- Resource Groups
- Resources
- Service Principals



Interno

Lab 2: Building Provisioning Structures Using Variables



38

Interno

THANK YOU!

CONTENT:
FM ACADEMY
INSTRUCTOR: DYEGO SOUZA
RAMEHX@YAHOO.COM.BR
HTTPS://WWW.LINKEDIN.COM/IN/DYEGODESOUZA/

Interno